5.3 自动化实战案例

> **生成模型**：OpenAI GPT-5.3 Codex (`openai/gpt-5.3-codex`)
> **Token 消耗**：输入 ~26k tokens，输出 ~6.2k tokens（估算）

前两节我们讲了机制，这一节直接上“能跑、能用、能省时间”的实战。目标不是做炫技 demo，而是给你 5 套能落地在学生/开发者日常里的自动化模板。你可以先抄一套跑通，再替换成自己的渠道和任务描述。

每个案例都包含：场景目标、Cron 表达式、 `openclaw.json` 片段、预期行为。

5.3.1 案例一：每日天气报告

场景目标

每天早上 8 点自动生成天气简报，包含温度、降雨概率、穿衣建议、通勤提醒，发到 Telegram。

Cron 表达式

`0 8 * * *` （每天 8:00）

openclaw.json 配置片段

```
{
  cron: {
    enabled: true,
    sessionRetention: "24h"
  },
  jobs: [
    {
      name: "Daily Weather Brief",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "0 8 * * *",
        tz: "Asia/Shanghai"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "查询今天杭州天气，输出天气概况、最高/最低温、降雨概率、穿衣建议和一句通勤提醒，控制
      },
      delivery: {
        mode: "announce",
        channel: "telegram",
        to: "123456789",
        bestEffort: true
      }
    }
  ]
}
```

预期行为

每天到点后，你会收到一段可直接读完的天气摘要，不用再打开多个 App 拼信息。遇到极端天气时，Agent 通常会把提醒写得更具体（比如降雨时间窗、体感变化、出行注意事项）。

5.3.2 案例二：服务器健康监控

场景目标

每 15 分钟巡检服务器健康状态：磁盘、内存、CPU、关键服务、失败的 systemd 单元；异常时先诊断，再执行保守修复建议，结果发到 Slack。

Cron 表达式

`*/15 * * * *` （每 15 分钟）

openclaw.json 配置片段

```
{
  cron: {
    enabled: true,
    maxConcurrentRuns: 2,
    sessionRetention: "12h"
  },
  jobs: [
    {
      name: "Server Health Check",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "*/15 * * * *"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "检查磁盘、内存、CPU负载、nginx、postgres和失败的systemd服务；发现异常先解释原因
      },
      delivery: {
        mode: "announce",
        channel: "slack",
        to: "channel:C0123456789",
        bestEffort: true
      }
    }
  ]
}
```

预期行为

正常时你会收到简短“绿色状态”摘要；异常时会收到“问题 -> 原因 -> 建议动作”的结构化信息。若可自动处理，Agent 会先做低风险处置；高风险动作应保留人工确认。

5.3.3 案例三：每日新闻摘要

场景目标

每天 7:30 自动抓取 AI、开源、开发者社区新闻，生成 5 条去水版摘要，发到 Discord。

Cron 表达式

`30 7 * * *` （每天 7:30）

openclaw.json 配置片段

```
{
  cron: {
    enabled: true,
    sessionRetention: "24h"
  },
  jobs: [
    {
      name: "Daily News Digest",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "30 7 * * *",
        tz: "Asia/Shanghai"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "抓取今天值得关注的AI、开源和编程新闻，筛掉营销稿，输出5条摘要。每条包含：标题、核心
      },
      delivery: {
        mode: "announce",
        channel: "discord",
        to: "channel:123456789012345678",
        bestEffort: true
      }
    }
  ]
}
```

预期行为

每天早上你收到的是“已经压缩过的信息包”，而不是一堆原始链接。特别适合课程学习和晨会准备：先用摘要做筛选，再决定哪些值得深读。

5.3.4 案例四：自动化社交媒体发帖

场景目标

工作日中午 12 点从选题池生成一条短帖，自动投递到内容系统 webhook；如果你希望更稳，也可以先发给自己审核。

Cron 表达式

`0 12 * * 1-5` （工作日 12:00）

openclaw.json 配置片段

```
{
  cron: {
    enabled: true,
    webhookToken: "cron-webhook-token",
    sessionRetention: "48h"
  },
  jobs: [
    {
      name: "Social Draft Generator",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "0 12 * * 1-5",
        tz: "Asia/Shanghai"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "根据今天的选题池写一条120字以内中文短帖，语气自然、非营销腔；附3个备选标题和1条配图
      },
      delivery: {
        mode: "webhook",
        to: "https://content.example.com/openclaw/social-draft"
      }
    }
  ]
}
```

预期行为

到点后内容系统会收到结构化草稿。你可以在后续流程接“人工审核 -> 排期 -> 发布”。如果暂时不想直连发布系统，把 `delivery.mode` 改成 `announce` 发给 Telegram/Slack 先看稿即可。

5.3.5 案例五：定时代码备份

场景目标

每天晚上 11 点检查项目仓库状态、执行备份脚本、汇总结果并通知你，避免“写了很多但没备份”。

Cron 表达式

`0 23 * * *` （每天 23:00）

openclaw.json 配置片段

```
{
  cron: {
    enabled: true,
    sessionRetention: "72h"
  },
  jobs: [
    {
      name: "Code Backup",
      enabled: true,
      schedule: {
        kind: "cron",
        expr: "0 23 * * *",
        tz: "Asia/Shanghai"
      },
      sessionTarget: "isolated",
      payload: {
        kind: "agentTurn",
        message: "检查指定代码目录的Git状态，执行备份脚本或归档命令，输出备份结果、异常项和风险提示；
      },
      delivery: {
        mode: "announce",
        channel: "telegram",
        to: "123456789",
        bestEffort: true
      }
    }
  ]
}
```

预期行为

你每天会收到“哪些仓库有变化、哪些已备份、哪些失败”的摘要，不再靠记忆做备份。

5.3.6 把这 5 套方案用稳的实践建议

1. **先低风险闭环**：先做“自动收集 + 自动摘要 + 人工确认”，再逐步放权到自动执行。
2. **高频任务用隔离会话**：保持主会话干净，排错也更直观。

3. **prompt 写清楚输出格式**：比如条数、字数、字段结构，不然结果容易飘。
4. **固定投递目标**：关键任务别依赖 `last` ，显式写 `channel` 和 `to` 。
5. **上线前强制试跑**：用 `openclaw cron run <job-id> --force` 验证表达式、权限、渠道。
6. **关键流程加兜底**：事件驱动流程可再加一个日终 Cron 补扫，防止漏处理。

本节小结

这 5 个案例覆盖了自动化最常见的价值面：信息推送（天气、新闻）、运维值班（健康监控）、内容生产（社媒草稿）、开发保障（代码备份）。它们的共通点是：都能通过“Cron 触发 + Agent 执行 + Delivery 投递”落地。你不需要一次做很重，先挑一个最痛的重复动作跑起来，通常一两天内就会明显感觉到 OpenClaw 从“会聊天”变成了“会干活”。